조건문과 반복문

컴퓨터학부 박한영

목차

01
논리값과 연산자

02
조건문

03
반복문

C언어의 논리값

- Java, Python, C++ 등의 언어에는 부울 타입이 있어 이를 통해 참/거짓을 판별

```java
boolean val1 = true;
boolean val2 = false;
// case of java
```

```python
bool1 = True
bool2 = False
# case of python
```

- 그러나 C언어는 기본적으로 부울 타입이 없기에, 컴파일러에서 0이면 false,

  나머지 경우에 대해서는 모두 true로 처리함.

```c
int a;
scanf("%d", &a);
if(a == 1)
    printf("True");
else if(a == 0)
    printf("False");
else
    printf("True");
```

비교/논리/삼항 연산자

- 연산자의 종류
  - 비교 연산자: ==(동등), !=(다름), >, >=, <, <=
  - 논리 연산자: &&(AND), ||(OR), !(NOT)
  - 삼항 연산자: (조건식) ? (참일 때) : (거짓일 때)

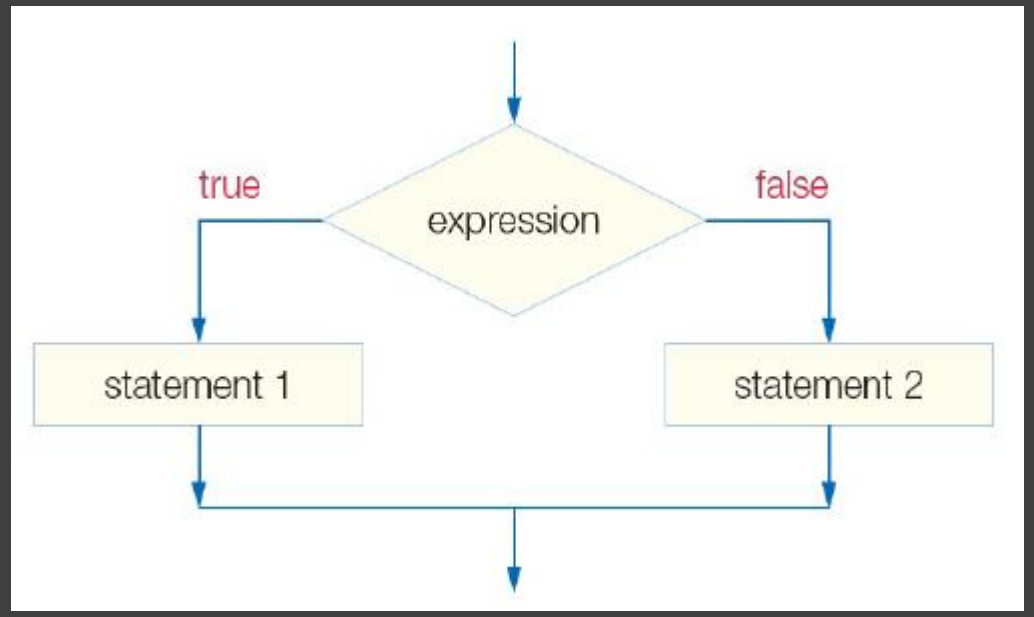
- C에서의 논리 연산은 그 결과가 참일 경우 1(true), 거짓일 경우 0(false)를 반환
- 조건문의 조건식에 다양한 연산자를 활용하여 코드를 간략화할 수 있음!

```c
if(hour >= 9) {
    if(hour <= 10) {
        printf("1교시");
    }
}
```

```c
if(hour >= 9 && hour <= 10) {
    printf("1교시");
}
```

조건문

- 조건문은 상황에 따라 다른 동작을 수행하기 위해 사용
- 조건식의 참/거짓에 따라 보통 다른 명령어들을 수행
- 조건식은 비교 연산자와 논리 연산자를 이용해 작성
- C언어에서는 if문과 switch문을 조건문으로 사용



if문

- 조건식이 참이면 주어진 명령문을 수행
- if, else if, else로 나눌 수 있음
- 명령문이 한 줄일 경우 중괄호 생략 가능!

```c
// 1. if 단일
if(condition)
    statement;

// 2. if | else
if(condition)
    statement;
else
    statement;

// 3. if | else if | else
if(condition)
    statement;
else if(condition)
    statement;
else
    statement;
```

```c
if(score >= 90) {
    printf("A입니다.");
}
else if(score >= 80) {
    printf("B입니다.");
}
else if(score >= 70) {
    printf("C입니다.");
}
else if(score >= 60) {
    printf("D입니다.");
}
else {
    printf("F입니다.");
}
```

단축 평가

- (조건1 || 조건2 …)에서 조건1이 참이면
  -> OR 연산은 하나만 참이어도 참이므로 연산 종료
- (조건1 && 조건2 …)에서 조건1이 거짓이면
  -> AND 연산은 하나만 거짓이어도 거짓이므로 연산 종료

예시: 0으로 나누기 방지

```
if(x != 0 && val / x > 0)
    statement;
```

➢ 이경우 x != 0이 먼저 계산되므로 val을 0으로 나누는 걸 방지할 수 있음!

```
if(val / x > 0 && x != 0)
    statement;
```

➢ 이경우 val / x가 먼저 계산되므로 x = 0인 경우 zero division error가 발생함!

switch문

- 주어진 값의 case에 따라 해당 명령어를 수행하는 조건문
- case 이외의 경우는 default 부분의 명령어를 수행
- case에 변수가 올 수 없으며, 정수형인 상수값만 가능

```c
int io;
scanf("%d", &io);
if(io == 0)
    printf("입력");
else if(io == 1)
    printf("출력");
else
    printf("에러");
return 0;
```

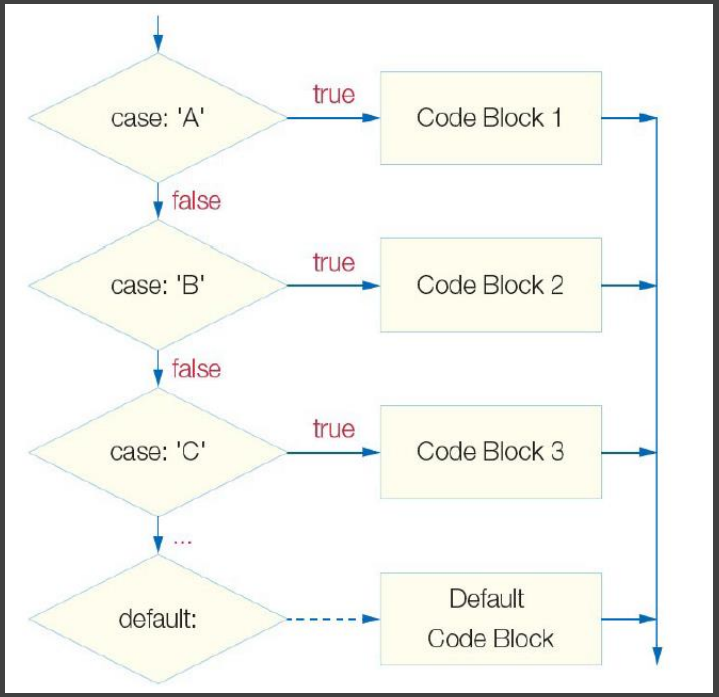
```c
int io;
scanf("%d", &io);
switch(io) {
    case 0:
    printf("입력");
    break;
    case 1:
    printf("출력");
    break;
    default:
    printf("에러");
    break; // 가장 뒤의 경우 생략 가능
}
```

switch문

- 여러 case에 대해 같은 명령어를 수행하게 할 수도 있음
- break를 쓰지 않는 경우 아래의 명령어들을 계속 수행

```c
char ch;
scanf("%c", &ch);
if(ch == 'A' || ch == 'a')
    printf("우수");
else if(ch == 'B' || ch == 'b')
    printf("우등");
else if(ch == 'C' || ch == 'c')
    printf("보통");
else
    pritnf("F");
```

```c
char ch;
scanf("%c", &ch);
switch(ch) {
    case 'A':
    case 'a':
    printf("우수");
    break;
    case 'B':
    case 'b':
    printf("우등");
    // 만약 break가 없다면??
    case 'C':
    case 'c':
    printf("보통");
    break;
    default:
    printf("F");
    break;
}
```



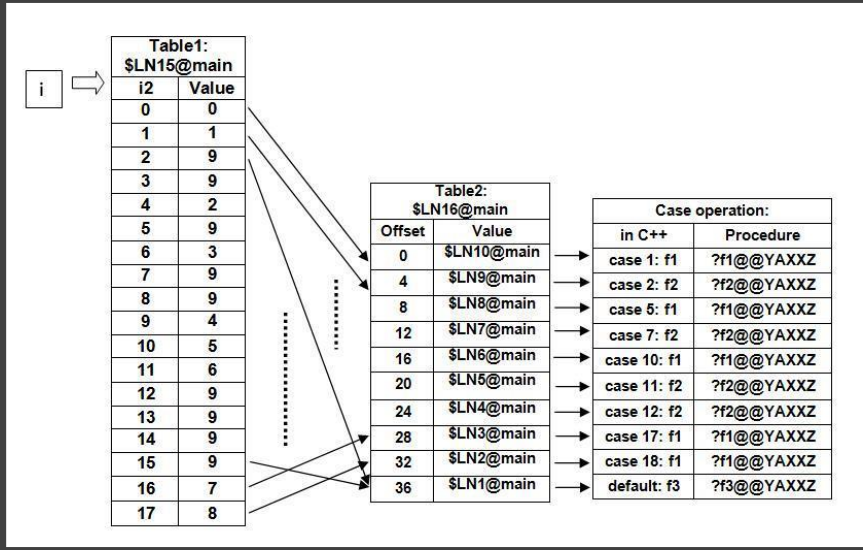
switch문의 흐름

조건문의 연산 속도

- if문의 경우 모든 조건에 대해 비교 연산을 수행
- switch문의 경우 컴파일러에 의해 초기부터 case에 따라 어떤 연산을 수행할 지 분기가 나눠진 jump table이 생성됨
  - 따라서 case의 값에는 변수가 올 수 없으며, 들어온 case를 보고 바로 해당 분기로 점프하기 때문에 if문처럼 분기마다 연산 x

조건문의 연산 속도

- 정리
  - if는 조건을 만날 때마다 검사하고, switch는 값만 받고 해당 분기로 가서 실행
  - 그렇기에 연산속도는 switch가 더 빠름
  - 또한case 값의 크기가 작고, 값이 순차적으로 정렬되어 있으며, 그 값 간의 차이가 크지 않을수록 switch를 써서 연산 시간을 단축하는 효과가 극대화됨



jump table

goto문(설명 x)

- goto + '라벨이름' 을 만날 경우, '라벨이름:' 으로 점프하는 구조

```c
#include <stdio.h>

int main() {
    int answer;
    printf("c언어 강의를 구매하시겠습니까? (1: 예, 2: 아니오): ");
    scanf("%d", &answer);
    if (answer == 2) {
        goto Quit;
    } else {
        printf("1: 초급 과정 구매, 2: 중급 과정 구매, 3: 고급 과정 구매, 4: 구매 취소: ");
        scanf("%d", &answer);
        if (answer == 4) {
            printf("구매를 취소하셨습니다.\n");
        } else {
            printf("구매가 완료되었습니다.\n");
        }
    }

    Quit:
    printf("강의 구매 프로그램을 종료합니다.\n");
    return 0;
}
```

goto문(설명 x)

- goto의 단점
  - goto를 쓰면 프로그램의 흐름이 여기저기 왔다갔다하여 코드의 흐름 구조를 파악하기 힘들어진다.
  - 함수의 스택을 날리고, 바로 해당 위치로 이동해버려 프로그램의 구조적 흐름이 꼬인다.
    - 스파게티 면처럼 꼬인 코드라고 하여 스파게티 코드라고 불림
    - 지양해야할 코딩 스타일

goto문(설명 x)

- 그럼에도 goto를 쓰면 좋은 경우?
  - ➢ 다중 반복 탈출하기
    - ➢ 물론 이 코드도 goto없이 구현이 가능하지만, 다중 반복 탈출 여부를 알 수 있는 변수를 추가해야 하는 것이 번거로울 때 다음과 같이 응용할 수 있다.

```c
#include <stdio.h>

int main() {
    int quit = 0;

    for (int i = 1; i <= 9; i++) {
        printf("| ");
        for (int j = 1; j <= 9; j++) {
            if (i == 4 && j == 9) {
                quit = 1;
                break;
            }
            printf("%d * %d = %2d | ", i, j, i * j);
        }
        if (quit)
            break;
        printf("\n");
    }
    return 0;
}
```

goto 사용x

```c
#include <stdio.h>

int main() {
    for (int i = 1; i <= 9; i++) {
        printf("| ");
        for (int j = 1; j <= 9; j++) {
            if (i == 4 && j == 9) {
                goto EXIT;
            }
            printf("%d * %d = %2d | ", i, j, i * j);
        }
        printf("\n");
    }
    EXIT:
    return 0;
}
```

goto 사용

for문

- 초기식; 조건식; 증감식의 구조를 가지고 있는 반복문
- 초기식을 선언한 후 조건식이 참이면 명령문을 실행하고 증감식을 수행한 후 다시 조건식에서 검사를 반복하며 실행되는 구조

for (초기식; 조건식; 증감식) { 명령문; }

- ➢ 초기식: for문 들어가기 전 한 번만 실행
- ➢ 조건식: 매 루프마다 조건식을 검사하고, 참인 경우 명령문을 수행
- ➢ 증감식: 루프가 끝날 때마다 실행됨

```
for(int i = 0; i < n; i++)
    printf("%d\n", i);
```

i++과 ++i의 차이는?

for 루프는 반복이 명확할 때 사용!

while문

- 조건식이 참인 동안 명령문을 수행하고, 루프가 끝날 때마다 다시 조건식을 검사하여 반복하는 구조

while(조건식) { 명령문; }

➢ 조건식: 매 루프마다 조건식을 검사하고, 참인 경우 명령문을 수행

```
int n;
scanf("%d", &n);
while(n < 100) {
    n *= 2;
    printf("%d\n", n);
}
```

while 루프는 반복이 명확하지 않을 때 사용!

do-while문

- while문과 동일하게 조건식이 참인 동안 명령문을 수행하지만, 맨 처음의 명령문 실행을 보장하는 것이 차이!

```
do {
    printf("처음은 무조건 실행");
} while(0);
```

do { 명령문; } while(조건식);

➢ 조건식: 일단 명령문을 한 번 실행하고, 이후 매 루프마다 조건식을 검사

반복 흐름 제어

- break: 반복문에서 break를 만나면 그 이후의 모든 코드를 무시하고 루프를 탈출한다.
- continue: 반복문에서 continue를 만나면 그 이후의 모든 코드를 무시하고 다시 루프를 수행한다.

```c
for(int i = 1; i <= 100; i++) {
    if(i % 10 == 0)
        continue;
    printf("%d ", i);
}
```

<continue>

```c
for(int i = 1; i <= 100; i++) {
    if(i % 10 == 0)
        break;
    printf("%d ", i);
}
```

<break>

THANK YOU!